



Ojo ERC4626 Price Feed

Security Review



Disclaimer

Security Review

Ojo ERC4626 Price Feed



Disclaimer

The ensuing audit offers no assertions or assurances about the code's security. It cannot be deemed an adequate judgment of the contract's correctness on its own. The authors of this audit present it solely as an informational exercise, reporting the thorough research involved in the secure development of the intended contracts, and make no material claims or guarantees regarding the contract's post-deployment operation. The authors of this report disclaim all liability for all kinds of potential consequences of the contract's deployment or use. Due to the possibility of human error occurring during the code's manual review process, we advise the client team to commission several independent audits in addition to a public bug bounty program.

Table of Contents

Security Review

Ojo ERC4626 Price Feed



Table of Contents

Disclaimer	3
Summary	7
Scope	9
Methodology	11
Project Dashboard	13
Risk Section	16
Findings	18
3S-Ojo-N01	18

Summary Security Review

Ojo ERC4626 Price Feed



Summary

Three Sigma audited Ojo's ERC4626 Price Feed in a 0.4 person week engagement. The audit was conducted on 23/06/2025.

Protocol Description

ERC4626PriceFeed is an adapter contract that wraps ERC4626 vaults to expose their share-to-asset conversion rates through the Chainlink AggregatorV3Interface, enabling vault prices to be consumed by any protocol expecting Chainlink price feeds. The contract implements all required aggregator methods by calling `vault.convertToAssets(1e18)` to return the asset value of one share, using static round IDs and block timestamps for feed metadata.

Scope Security Review

Ojo ERC4626 Price Feed



Scope

Filepath	nSLOC
src/ERC4626PriceFeed.sol	71
src/ERC4626PriceFeedFactory.sol	20
Total	91

Methodology Security Review

Ojo ERC4626 Price Feed



Methodology

To begin, we reasoned meticulously about the contract's business logic, checking security-critical features to ensure that there were no gaps in the business logic and/or inconsistencies between the aforementioned logic and the implementation. Second, we thoroughly examined the code for known security flaws and attack vectors. Finally, we discussed the most catastrophic situations with the team and reasoned backwards to ensure they are not reachable in any unintentional form.

Taxonomy

In this audit, we classify findings based on Immunefi's [Vulnerability Severity Classification System \(v2.3\)](#) as a guideline. The final classification considers both the potential impact of an issue, as defined in the referenced system, and its likelihood of being exploited. The following table summarizes the general expected classification according to impact and likelihood; however, each issue will be evaluated on a case-by-case basis and may not strictly follow it.

Impact / Likelihood	LOW	MEDIUM	HIGH
NONE	None		
LOW	Low		
MEDIUM	Low	Medium	Medium
HIGH	Medium	High	High
CRITICAL	High	Critical	Critical

Project Dashboard

Security Review

Ojo ERC4626 Price Feed



Project Dashboard

Application Summary

Name	Ojo ERC4626 Price Feed
Repository	https://github.com/ojo-network/yield-risk-engine
Commit	e112ea9
Language	Solidity
Platform	Ethereum

Engagement Summary

Timeline	23/06/2025
Nº of Auditors	2
Review Time	0.4 person weeks

Vulnerability Summary

Issue Classification	Found	Addressed	Acknowledged
Critical	0	0	0
High	0	0	0
Medium	0	0	0
Low	0	0	0
None	1	0	1

Category Breakdown

Suggestion	1
Documentation	0
Bug	0
Optimization	0
Good Code Practices	0

Risk Section Security Review

Ojo ERC4626 Price Feed



Risk Section

No risk has been identified.

Findings

Security Review

Ojo ERC4626 Price Feed



Findings

3S-Ojo-N01

Inconsistent access control design allows unrestricted price feed creation

Id	3S-Ojo-N01
Classification	None
Category	Suggestion
Status	Acknowledged

Description

The **ERC4626PriceFeedFactory::createPriceFeed** function creates clones of the price feed implementation and stores them in the **priceFeedsByVault** mapping without any restrictions. While the function validates that the vault address is not zero, it does not restrict who can call it or which vaults can have price feeds created for them. The contract imports Ownable, but the **onlyOwner** modifier is not used, resulting in inconsistent access control design.

```
function createPriceFeed(address vault, string memory description) external returns
(address priceFeed) {
    require(vault != address(0), "zero vault address");
    priceFeed = implementation.clone();
    ERC4626PriceFeed(priceFeed).initialize(vault, description);
    priceFeedsByVault[vault].push(priceFeed);
    emit PriceFeedCreated(vault, priceFeed);
}
```

This design allows malicious actors to spam the factory with useless price feed creations, bloating the **priceFeedsByVault** mapping with potentially misleading or unnecessary entries.

Recommendation

Add the **onlyOwner** modifier to the **createPriceFeed** function to restrict price feed creation to the contract owner. If the lack of access control is intentional, consider removing the Ownable import to avoid confusion and make the design choice explicit.